

CS330 Operating Systems and Lab.

Synchronization

Spring 2026

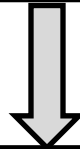


Instructor : Insik Shin

In previous lectures: Multiprogramming

Goals:

- Multiprogramming: Run multiple applications concurrently



Solution:

Process & Thread: unit of execution and allocation

- Virtual Machine abstraction: gives **an illusion that it owns a machine** (i.e., CPU, Memory, and IO device multiplexing)



App 1's results

when it uses the
whole machine
alone



App 1's results

when it shares the
machine (using only a
portion of the machine)

In previous lectures: Multiprogramming

Goals:

- Multiprogramming: Run multiple applications concurrently



Solution:

Process & Thread: unit of execution and allocation

- Virtual Machine abstraction: gives **an illusion that it owns a machine** (i.e., CPU, Memory, and IO device multiplexing)



Deterministic results?

Logical results

Performance

In previous lectures: Multiprogramming

Goals:

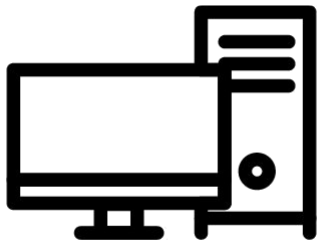
- Multiprogramming: Run multiple applications concurrently



Solution:

Process & Thread: unit of execution and allocation

- Virtual Machine abstraction: gives **an illusion that it owns a machine** (i.e., CPU, Memory, and IO device multiplexing)



App results

when it uses
multiple threads



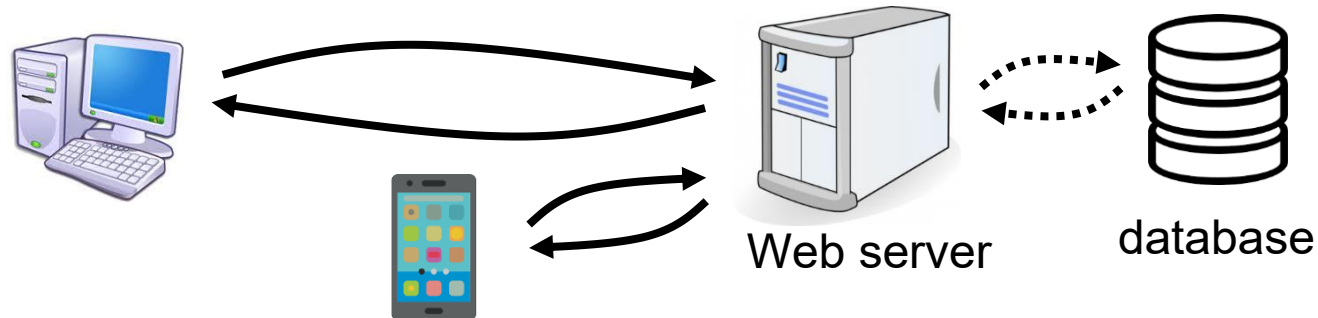
App results

when it uses a
single thread

Concurrent Threads - Why?

- People cooperate; computers help/enhance people's lives, so computers must cooperate
- Advantage 1: Share resources
 - One computer, many users
 - One bank balance, many online banking clients
 - Embedded systems (robot control: coordinate arm & hand)
- Advantage 2: Speedup
 - Overlap I/O and computation
 - Multiprocessors – chop up program into parallel pieces
- Advantage 3: Modularity
 - Chop a large problem up into simpler pieces
 - To compile, for instance, gcc calls cpp | cc1 | cc2 | as | ld
 - Makes system easier to extend

Concurrent Threads - Web Server



- Multi-threaded version:

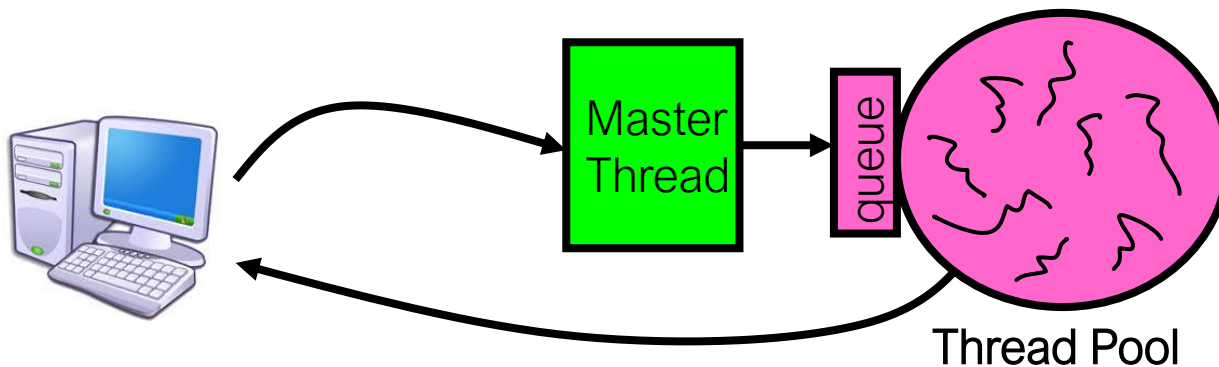
```
serverLoop() {  
    connection = AcceptConnection();  
    ThreadCreate(ServiceWebPage(), connection);  
}
```

- Advantages of threaded version:

- Can share file caches kept in memory, results of CGI scripts, other things
- Threads are *much* cheaper to create than processes, so this has a lower per-request overhead
- What if too many requests come in at once?

Thread Pools

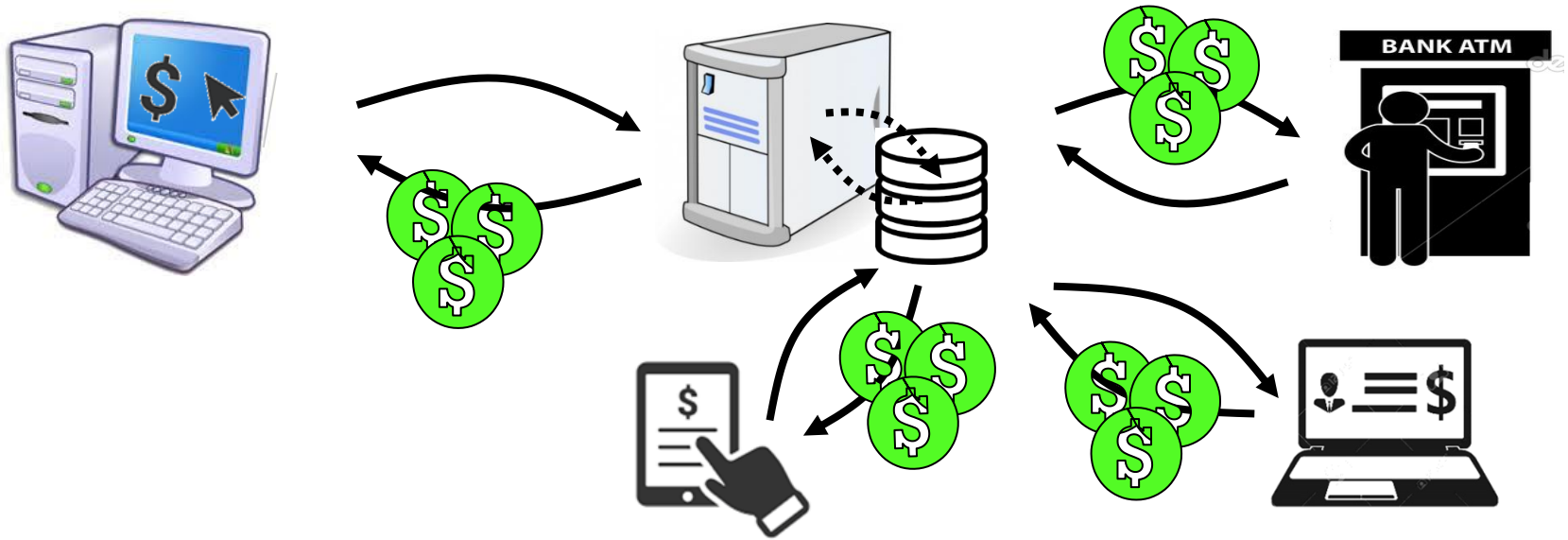
- Problem with previous version: Unbounded Threads
- Instead, allocate a bounded “pool” of threads, representing the maximum level of multiprocessing



```
master() {  
    allocThreads(slave, queue);  
    while(TRUE) {  
        con=AcceptConnection();  
        Enqueue(queue, con);  
        wakeUp(queue);  
    }  
}
```

```
slave(queue) {  
    while(TRUE) {  
        con=Dequeue(queue);  
        if (con==null)  
            sleepOn(queue);  
        else  
            ServiceWebPage(con);  
    }  
}
```

Online Banking Server



- Online banking server problem:
 - Service a set of requests
 - Do so without corrupting database
 - Don't hand out too much money

Online banking server example

- Suppose we wanted to implement a server process to handle requests from a client through the internet:

```
BankServer() {
    while (TRUE) {
        ReceiveRequest(&op, &acctId, &amount);
        ProcessRequest(op, acctId, amount);
    }
}

ProcessRequest(op, acctId, amount) {
    if (op == deposit) Deposit(acctId, amount);
    else if ...
}

Deposit(acctId, amount) {
    acct = GetAccount(acctId); /* may use disk I/O */
    acct->balance += amount;
    StoreAccount(acct);        /* Involves disk I/O */
}
```

- How could we speed this up?
 - More than one request being processed at once
 - Multiple threads (to overlap CPU computation and I/O)

Can Threads Help?

- One thread per request!
- While requests proceed to completion, thread switch happens when required (i.e., due to scheduling quantum (timer interrupt)):

```
Deposit(acctId, amount) {  
    acct = GetAccount(actId);    /* May use disk I/O */  
    acct->balance += amount;  
    /* compiled to three instructions */  
    StoreAccount(acct);          /* Involves disk I/O */  
}
```

- Unfortunately, shared state can get corrupted:

Thread 1

```
load r1, acct->balance
```

```
add r1, amount1  
store r1, acct->balance
```

Thread 2

```
load r1, acct->balance  
add r1, amount2  
store r1, acct->balance
```

Problem is at the lowest level

- Most of the time, threads are working on separate data, so scheduling doesn't matter:

Thread A

`x = 1;`

Thread B

`y = 2;`

- However, what if on shared data (initially, $y = 0$)?

Thread A

`x = 1;`

`x = y+1;`

Thread B

`y = 2;`

`y = y*2;`

- What are the possible values of x ?

Thread A

`x = 1;`

`x = y+1;`

Thread B

`y = 2;`

`y = y*2`

`x=1`

Problem is at the lowest level

- Most of the time, threads are working on separate data, so scheduling doesn't matter:

Thread A

$x = 1;$

Thread B

$y = 2;$

- However, what if on shared data (initially, $y = 0$)?

Thread A

$x = 1;$

$x = y + 1;$

Thread B

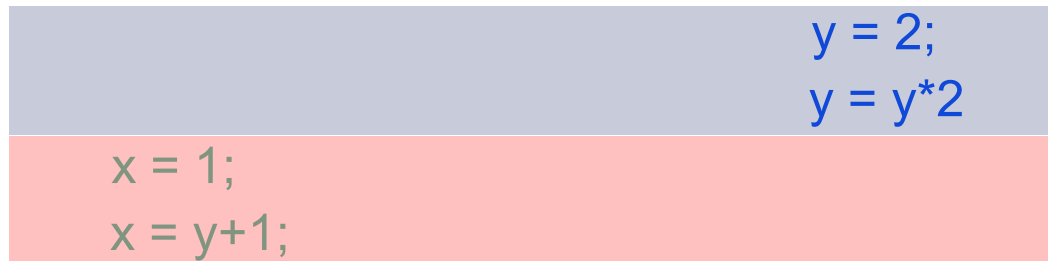
$y = 2;$

$y = y * 2;$

- What are the possible values of x ?

Thread A

Thread B



$x=5$

Problem is at the lowest level

- Most of the time, threads are working on separate data, so scheduling doesn't matter:

Thread A

$x = 1;$

Thread B

$y = 2;$

- However, what if on shared data (initially, $y = 0$)?

Thread A

$x = 1;$

$x = y + 1;$

Thread B

$y = 2;$

$y = y * 2;$

- What are the possible values of x ?

Thread A

Thread B



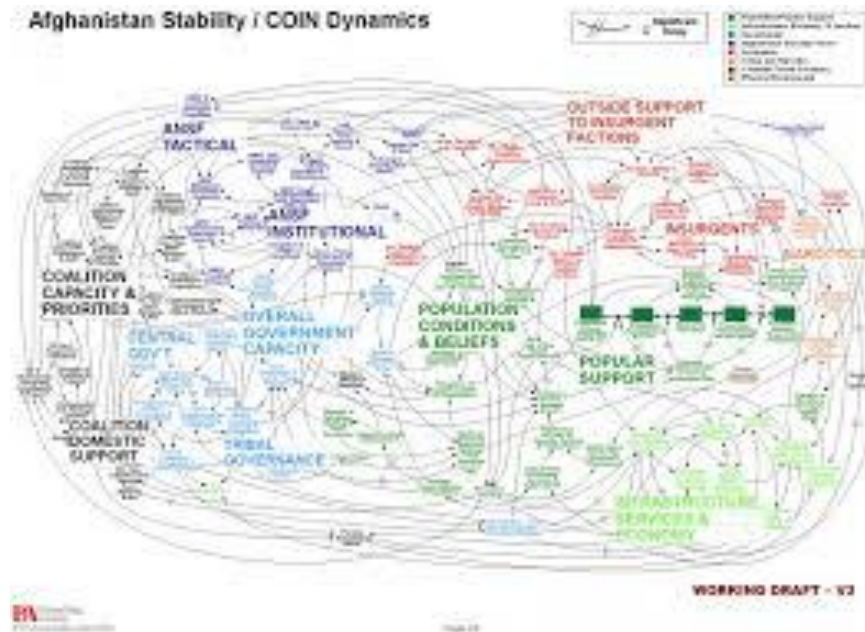
$x = 3$

Threads - Concurrency

- Concurrent threads are a very useful abstraction
 - Allow transparent overlapping of computation and I/O
 - Allow use of parallel processing when available
- Concurrent threads introduce problems when accessing shared data
 - Programs must be insensitive to arbitrary interleavings
 - Without careful design, shared variables can become completely inconsistent
- Next: deal with concurrency problems

Spaghetti Code

- a complex and tangled web of source code
- poorly structured and difficult to follow or maintain
 - overly complicated due to multiple patches, lack of clear structure, hard to trace the execution path
- It's named such since program flow is conceptually like a bowl of spaghetti, i.e., twisted and tangled



Correctness Requirements

- Threaded programs must work for all interleavings of thread instruction sequences
 - Cooperating threads inherently non-deterministic and non-reproducible
 - Really hard to debug unless carefully designed!
- Example: Therac-25
 - Machine for radiation therapy
 - Software control of electron accelerator and electron beam/Xray production
 - Software control of dosage
 - Software errors caused the death of several patients
 - A series of **race conditions** on shared variables and poor software design
 - “They determined that data entry speed during editing was the key factor in producing the error condition: If the prescription data was edited at a fast pace, the overdose occurred.”



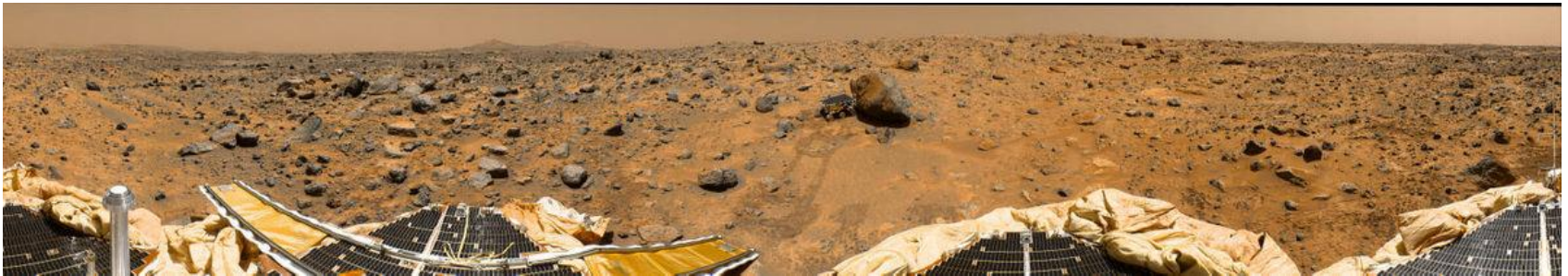
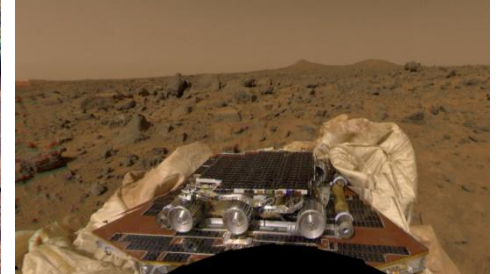
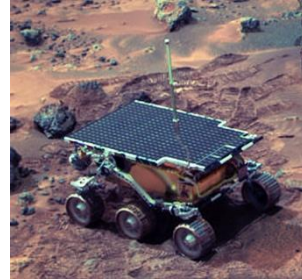
Correctness Requirements

- Example: **USA Northeast blackout of 2003**
 - Widespread power outage, **caused by the race condition**, throughout parts of the Northeastern and Midwestern United States
 - 55 million people suffered from the blackout
 - \$10 billion (10조원) of economic costs



Correctness Requirements

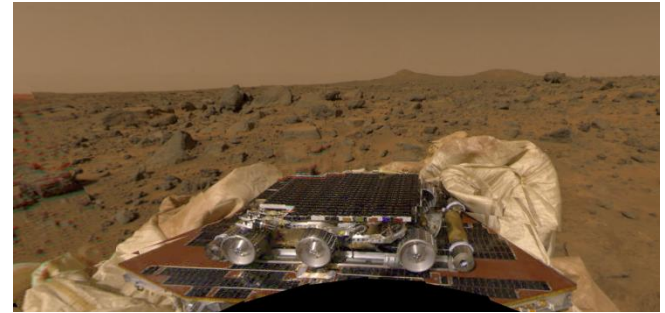
- Example: Mars Pathfinder
 - Spacecraft landed on Mars in 1997
 - surface operations, daily images
 - daily weather reports



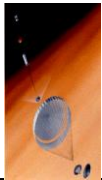
- "faster, better and cheaper" spacecraft
 - Aimed to show it is possible with three years for development and a cost under \$150 million (약1500억원)

Correctness Requirements

- Example: Mars Pathfinder
 - Spacecraft landed on Mars in 1997
 - surface operations, daily images
 - daily weather reports



landing



July 4th

successful operations



communication
errors



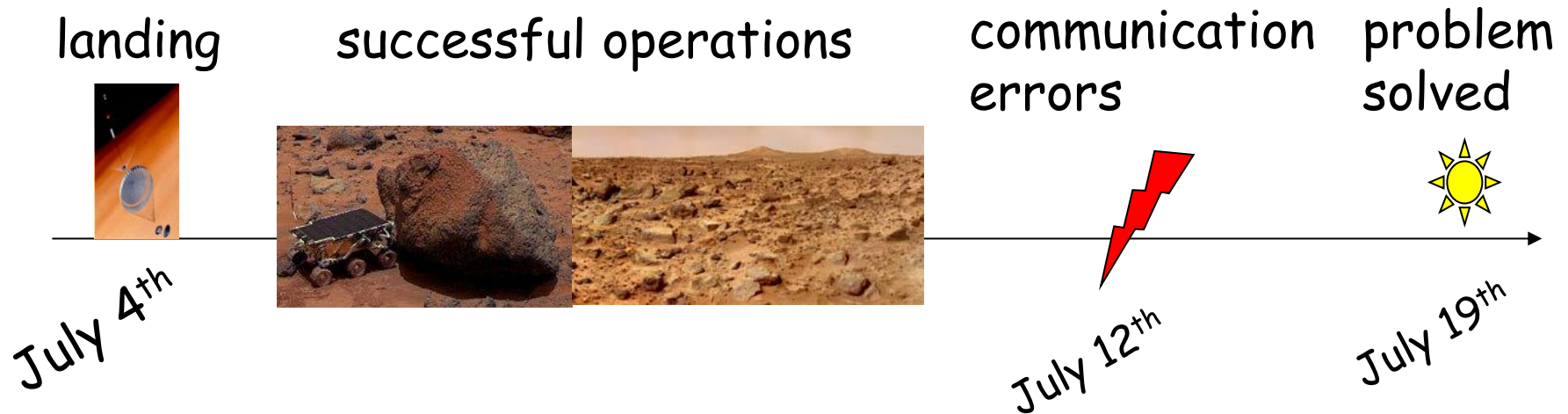
July 12th

- In a few days into the mission, it became out-of-order
 - It kept experiencing total system resets, doing nothing
 - Debugging has been going on the Earth with an exact spacecraft replica
 - It turned out that a **race condition** results in the problem

Priority Inversion & Mars Pathfinder



- Spacecraft landed on the Mars on July 4th, 1997 for the mission operations, including
 - surface operations
 - daily images of Mars
 - daily weather reports from the surface of Mars
- Out-of-control a week after landing, due to software problem



Where are we?

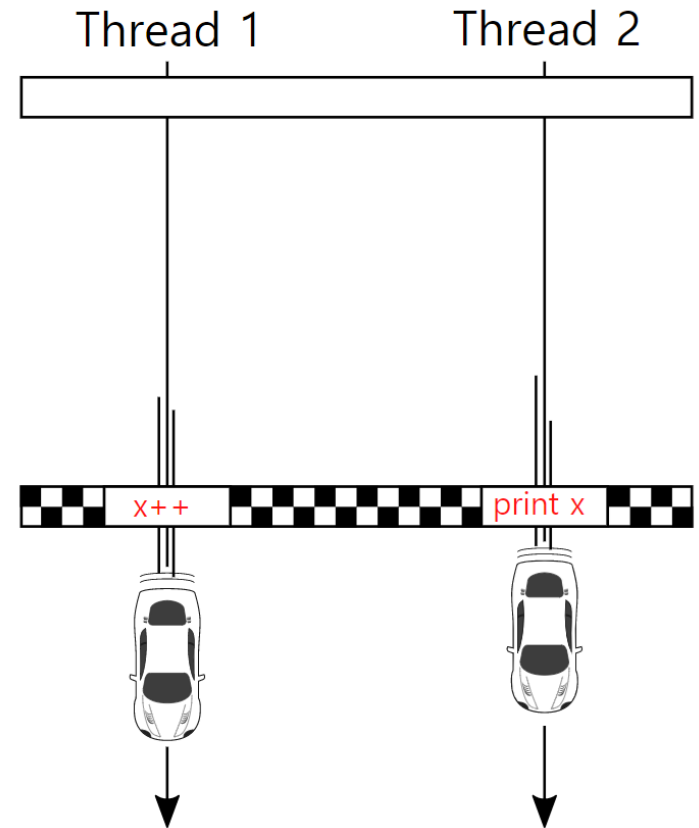
- Multi-program paradigm
- Process/thread for *great* abstraction
 - virtual machine (illusion of exclusive use)
 - transparent overlapping of I/O & computation, parallel execution
- Yet, deterministic results?
- Not in some cases, particularly, on race conditions
- How to resolve the problem with race conditions?
→ Synchronization

Threads - Concurrency

- Concurrent threads are a very useful abstraction
 - Allow transparent overlapping of computation and I/O
 - Allow use of parallel processing when available while maintaining sequential execution model
- Concurrent threads introduce problems when accessing shared data
 - **Programs must be insensitive to arbitrary interleavings**
 - Without careful design, shared variables can become completely inconsistent
- Next: deal with concurrency problems

Definitions

- **Race condition**
 - A situation where multiple processes or threads try to change shared data at the same time, and the outcome is dependent on the sequence of such accesses
 - Leads to unpredictable behavior and bugs that are difficult to reproduce and fix



More Definitions

- **Critical Section**: a piece of code that accesses a shared resource (producing a race condition)
- **Mutual Exclusion**: ensuring that only one thread executes critical section
 - One thread *excludes* the other(s) while doing its task
- **Synchronization**: coordination of concurrent processes or threads to ensure that they do not execute critical sections at the same time (using atomic operations to ensure cooperation between threads)
 - For now, only loads and stores are atomic
 - We'll show it is hard to build anything useful with only reads and writes

More Definitions

- **Lock**: prevents someone from doing something
 - Lock before entering critical section and before accessing shared data
 - Unlock when leaving, after accessing shared data
 - Wait if locked
 - Important idea: all synchronization involves waiting



Basic Structure for Synchronization

Thread 1



Sync. Entry

Critical
Section

Sync. Exit

Thread 2



Sync. Entry

Critical
Section

Sync. Exit

Synchronization

Programs	Shared Programs
Higher-level API	Locks Semaphores Monitors Send/Receive
Hardware	Load/Store Disable Ints Test&Set Comp&Swap

Motivation: “Too much milk”

- Great thing about OS's – analogy between problems in OS and problems in real life
 - Help you understand real life problems better
 - But, computers are much stupider than people
- Example: People need to coordinate:



Time	Person A	Person B
3:00	Look in Fridge. Out of milk	
3:05	Leave for store	
3:10	Arrive at store	Look in Fridge. Out of milk
3:15	Buy milk	Leave for store
3:20	Arrive home, put milk away	Arrive at store
3:25		Buy milk
3:30		Arrive home, put milk away

Too Much Milk

- What are the correctness properties for the “Too much milk” problem?
 - Never more than one person buys
 - Someone buys if needed
- How about an algorithm like this:

Thread A

```
if (noMilk) {  
    buy Milk;  
}
```

Thread B

```
if (noMilk) {  
    buy Milk;  
}
```

- Does this work?

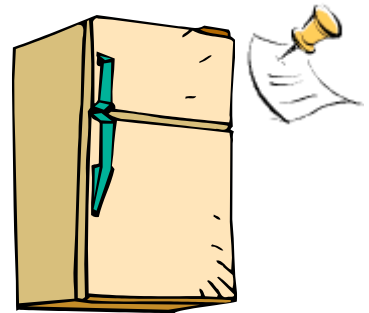
Too Much Milk: Correctness Properties

- Need to be careful about correctness of concurrent programs, since non-deterministic
 - Always write down **desired** behavior first
 - Impulse is to start coding first, then when it doesn't work, pull hair out
 - Instead, think first, then code
- Restrict ourselves to use only atomic memory (load and store) operations as building blocks

Too Much Milk: Solution #1

- Use a note to avoid buying too much milk:
 - Leave a note before buying (kind of “lock”)
 - Remove note after buying (kind of “unlock”)
 - Don't buy if note (wait)
- Suppose a computer tries this (remember, only memory read/write are atomic):

```
if (noMilk) {  
    if (noNote) {  
        leave Note;  
        buy milk;  
        remove Note;  
    }  
}
```



- Result?

Too Much Milk: Solution #1

- Still too much milk **but only occasionally!**

<u>Thread A</u>	<u>Thread B</u>
<pre>if (noMilk) if (noNote) {</pre>	
	<pre>if (noMilk) if (noNote) {</pre>
<pre> leave Note; buy milk; remove note; }</pre>	
<pre>}</pre>	<pre> leave Note; buy milk; ...</pre>

- Thread can get context switched after checking milk and note but before leaving note!
- Solution makes problem worse since fails **intermittently**
 - Makes it really hard to debug...
 - Must work despite what the thread dispatcher does!

Too Much Milk Solution #2

- How about early notes ?
 - We can leave notes before checking
- Algorithm looks like this:

Thread A

```
leave note A;  
if (noNote B) {  
    if (noMilk) {  
        buy Milk;  
    }  
}
```

Thread B

```
leave note B;  
if (noNote A) {  
    if (noMilk) {  
        buy Milk;  
    }  
}
```

- Does this work?

Too Much Milk Solution #2

- Possible for neither thread to buy milk!

Thread A

```
leave note A;
```

Thread B

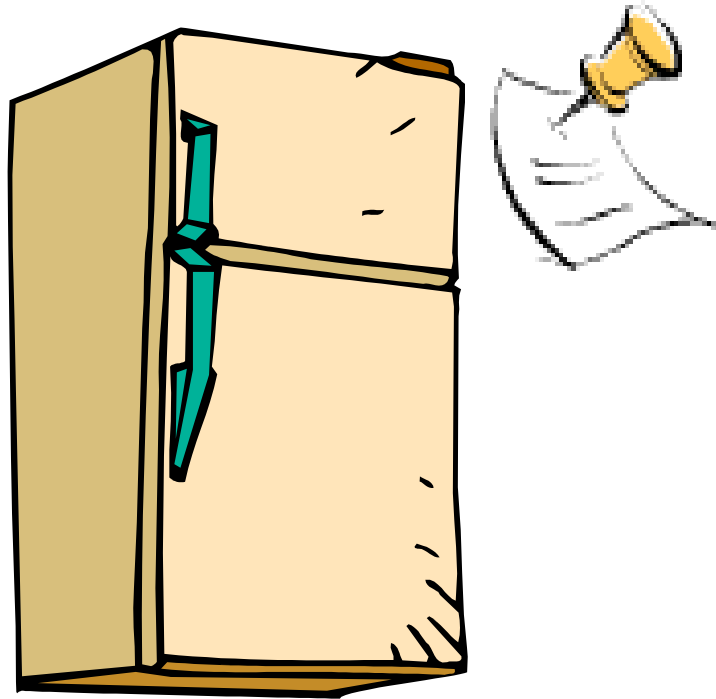
```
leave note B;  
if (noNote A) {  
    if (noMilk) {  
        buy Milk;  
    }  
}
```

```
if (noNote B) {  
    if (noMilk) {  
        buy Milk;  
    }  
    ...  
}
```

```
remove note B;
```

- Really insidious:
 - **Unlikely** that this would happen, but will at worse possible time

Too Much Milk Solution #2: problem!



- *I'm not getting milk, You're getting milk*
- This kind of lockup is called "starvation!"

Review: Too Much Milk Solution #3

- Here is a possible two-note solution:

<u>Thread A</u>	<u>Thread B</u>
leave note A;	leave note B;
while (note B) {\\X	if (noNote A) {\\Y
do nothing;	if (noMilk) {
}	buy milk;
if (noMilk) {	}
buy milk;	remove note B;
}	
remove note A;	

- Does this work? Yes. Both can guarantee that:
 - It is safe to buy, or
 - If the other will buy, it's ok to quit
- At X:
 - if no note B, safe for A to buy,
 - otherwise wait to find out what will happen
- At Y:
 - if no note A, safe for B to buy
 - Otherwise, A is either buying or waiting for B to quit

Review: Solution #3 discussion

- Our solution protects a single “Critical-Section” piece of code for each thread:

```
if (noMilk) {  
    buy milk;  
}
```

- Solution #3 works, but it's really unsatisfactory
 - Really complex – even for this simple example
 - Hard to convince yourself that this really works
 - A's code is different from B's – what if lots of threads?
 - Code would have to be slightly different for each thread
 - While A is waiting, it is consuming CPU time
 - This is called “busy-waiting”
- There's a better way
 - Have hardware provide better (higher-level) primitives than atomic load and store
 - Build even higher-level programming abstractions on this new hardware support

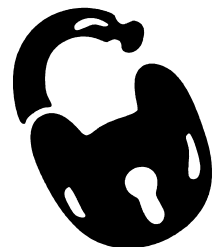
High-Level Picture

- The abstraction of threads is good:
 - Maintains sequential execution model
 - Allows simple parallelism to overlap I/O and computation
- Unfortunately, still too complicated to access state shared between threads
 - Consider “too much milk” example
 - Implementing a concurrent program with only loads and stores would be tricky and error-prone
- We’ll implement higher-level operations on top of atomic operations provided by hardware
 - Develop a “synchronization toolbox”
 - Explore some common programming paradigms



Lock

- **Lock:** a mechanism to control access to a shared resource by multiple processes or threads
 - Prevents someone from accessing something
 - Lock before entering critical section (e.g., before accessing shared data)
 - Unlock when leaving, after accessing shared data



Too Much Milk: Solution #4

- Suppose we have some sort of implementation of a lock (more in a moment).
 - `Lock.Acquire()` – wait until lock is free, then grab
 - `Lock.Release()` – unlock, waking up anyone waiting
 - These must be atomic operations – if two threads are waiting for the lock, only one succeeds to grab the lock
- Then, our milk problem is easy:

```
millock.Acquire();  
if (noMilk)  
    buy milk;  
millock.Release();
```

Then, how to implement a lock?

- Once again, section of code between `Acquire()` and `Release()` called a “**Critical Section**”

Naïve use of Interrupt Enable/Disable

- How can we build multi-instruction atomic operations?
 - Recall: dispatcher gets control in two ways.
 - Internal: Thread does something to relinquish the CPU
 - External: Interrupts cause dispatcher to take CPU
 - On a uniprocessor, we can avoid context-switching by:
 - Avoiding internal events (although virtual memory tricky)
 - Preventing external events by disabling interrupts
- Consequently, naïve Implementation of locks:

```
LockAcquire { disable Ints; }  
LockRelease { enable Ints; }
```

Locks

```
lock.Acquire();  
...  
critical section;  
...  
lock.Release();
```

Acquire() {
 disable interrupts;
}

Release() {
 enable interrupts;
}



If one thread in critical section, no other activity (including OS) can run!

Naïve use of Interrupt Enable/Disable: Problems

- **Can't let user do this!** Consider following:

```
LockAcquire();  
While(TRUE) {;}
```

- Real-Time system—no guarantees on timing!
 - Critical Sections might be arbitrarily long
- What happens with I/O or other important events?
 - “Reactor about to meltdown. Help?”



Better Implementation of Locks by Disabling Interrupts

- Key idea: maintain a lock variable and impose mutual exclusion only during operations on that lock variable

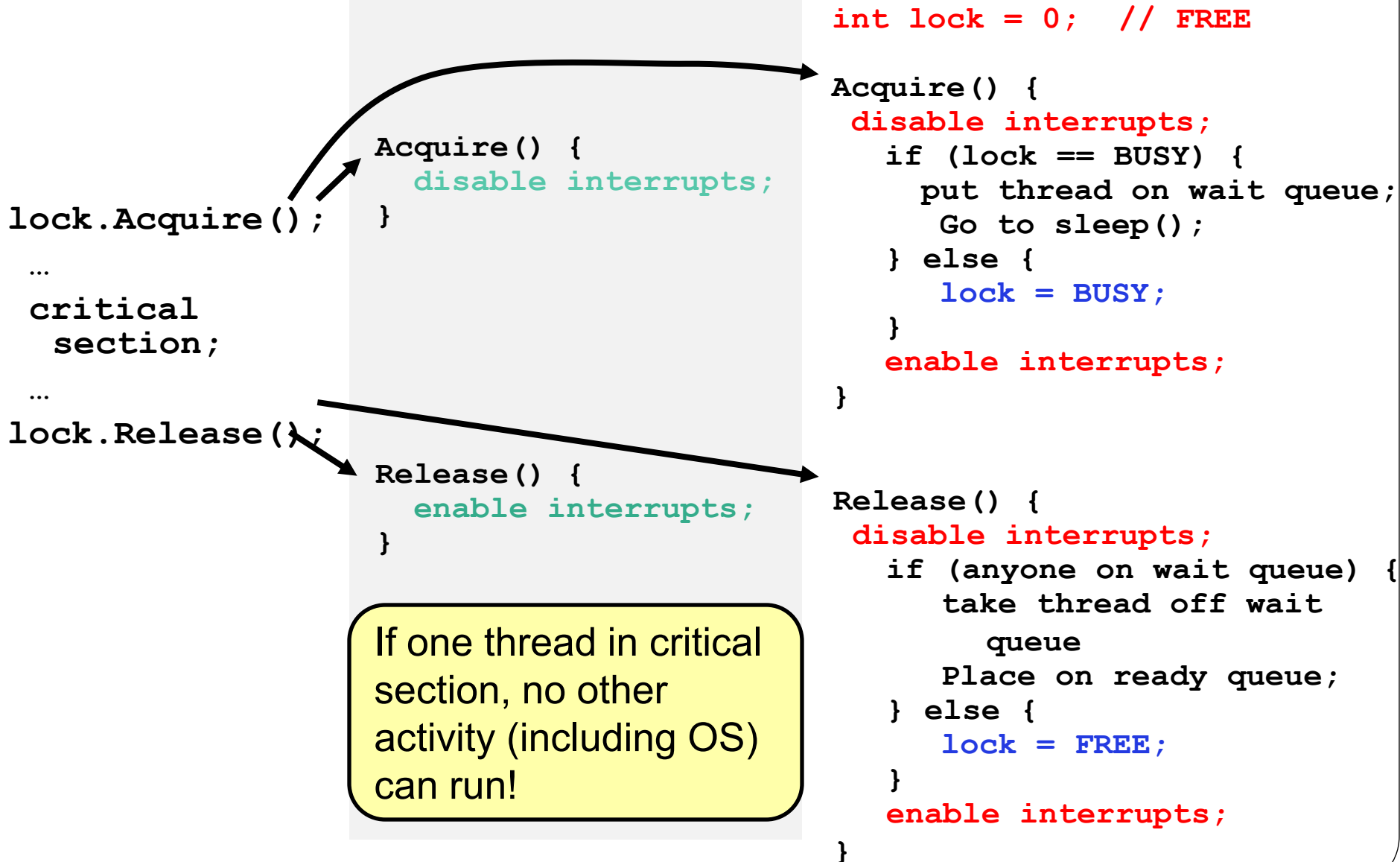
```
int lock = FREE;
```



```
Acquire() {  
    disable interrupts;  
    if (lock == BUSY) {  
        put thread on wait queue;  
        Go to sleep();  
    } else {  
        lock = BUSY;  
    }  
    enable interrupts;  
}
```

```
Release() {  
    disable interrupts;  
    if (anyone on wait queue) {  
        take thread off wait queue  
        Place on ready queue;  
    } else {  
        lock = FREE;  
    }  
    enable interrupts;  
}
```

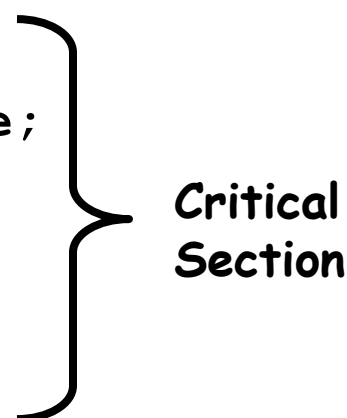
Locks



New Lock Implementation: Discussion

- Disable interrupts: avoid interrupting between checking and setting lock value
 - Otherwise two threads could think that they both have lock

```
Acquire() {  
    disable interrupts;  
    if (lock == BUSY) {  
        put thread on wait queue;  
        Go to sleep();  
    } else {  
        lock = BUSY;  
    }  
    enable interrupts;  
}
```



**Critical
Section**

- Note: unlike previous solution, critical section very short
 - User of lock can take as long as they like in their own critical section
 - Critical interrupts taken in time

Interrupt re-enable in going to sleep

- What about re-enabling interrupts when going to sleep?

Enable	Acquire() {	Release() {
Position	disable interrupts;	disable interrupts;
Enable	if (lock == BUSY) {	if (anyone on wait queue) {
Position	put thread on wait queue;	take thread off wait queue
	go to sleep();	Place on ready queue;
	} else {	} else {
	lock = BUSY;	lock = FREE;
	}	}
	enable interrupts;	enable interrupts;
	}	}

- Before putting thread on the wait queue?
 - Release can check the queue and will not wake up thread
- After putting the thread on the wait queue
 - Release puts the thread on the ready queue, but the thread still thinks it needs to go to sleep
 - Misses wakeup and still holds lock (deadlock!)

Summary

- Important concept: Atomic Operations
 - An operation that runs to completion or not at all
 - These are the primitives on which to construct various synchronization primitives
- Showed constructions of Locks using interrupts
 - Disabling of Interrupts
 - Must be very careful not to waste/tie up machine resources
 - Shouldn't disable interrupts for long
 - Key idea: Separate lock variable, use hardware mechanisms to protect modifications of that variable

Basic Structure for Synchronization

Thread 1



Sync. Entry

Critical
Section

Sync. Exit

Thread 2



Sync. Entry

Critical
Section

Sync. Exit

Synchronization

Programs	Shared Programs
Higher-level API	Locks Semaphores Monitors Send/Receive
Hardware	Load/Store Disable Ints Test&Set Comp&Swap

- We are going to implement various higher-level synchronization primitives using atomic operations
 - Everything is pretty painful if only atomic primitives are just memory instructions (load and store)
 - Need to provide primitives useful at user-level

Recap: Synchronization

- Needs to make a critical section atomic

```
lock.Acquire() ;  
...  
critical section;  
...  
lock.Release() ;
```

Recap: Locks

```
lock.Acquire();  
...  
critical section;  
...  
lock.Release();
```

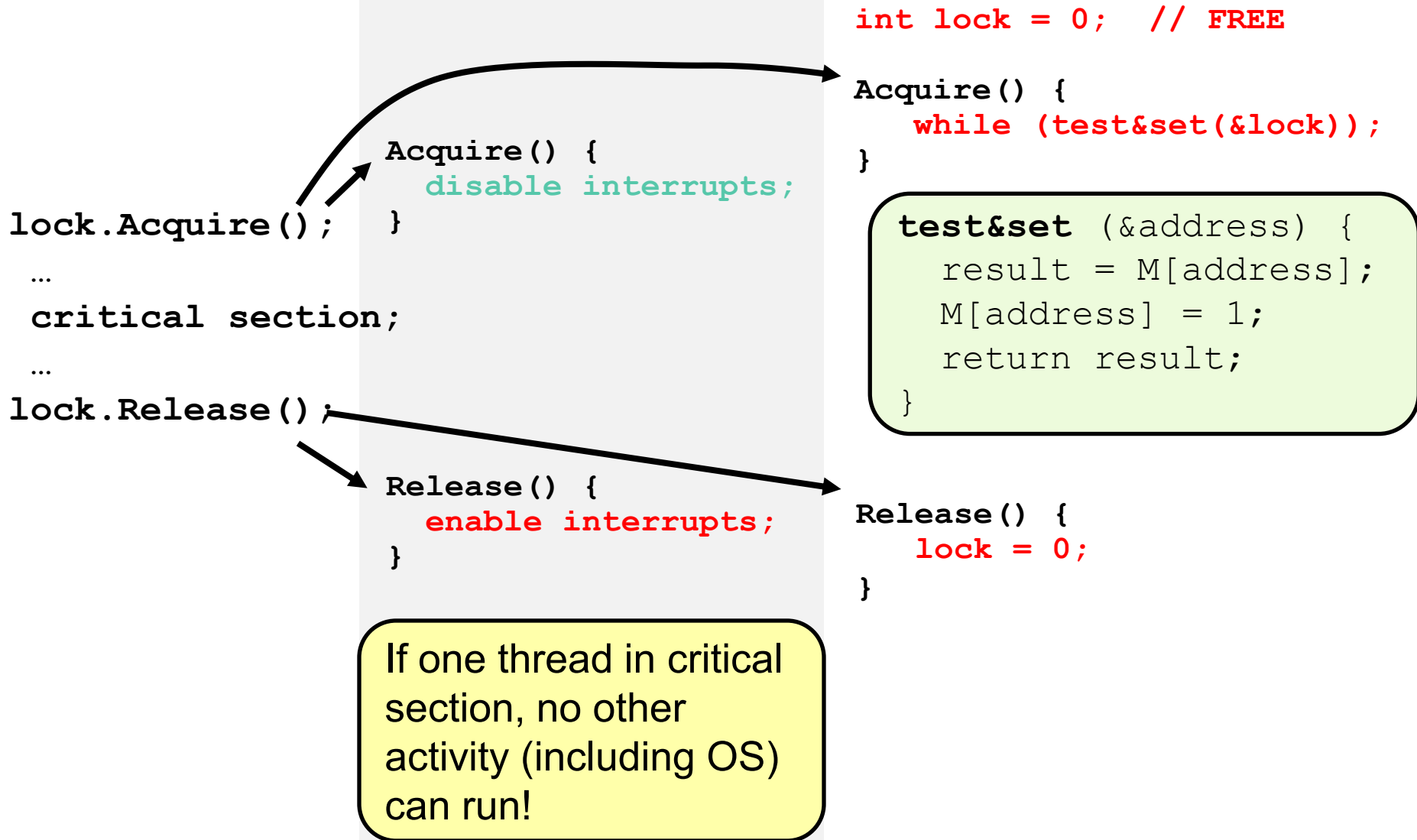
Acquire() {
 disable interrupts;
}

Release() {
 enable interrupts;
}



If one thread in critical section, no other activity (including OS) can run!

Recap: Locks



Hardware Support for Synchronization

- Hardware support for critical sections
 - Many modern systems provide hardware mechanisms to assist with synchronization
 - This support ensures that critical section code runs efficiently and safely
- Modern machines provide special atomic hardware instructions
 - Atomic = non-interruptible
 - Common atomic instructions
 - Test-and-Set: Reads a memory word and sets a value atomically
 - Swap: Exchanges the content of two memory words in a single step

TestAndSet Instruction

- The test-and-set instruction is an instruction used to write to a memory location and return its old value as a single atomic (i.e., non-interruptible) operation
- Definition:

```
boolean TestAndSet (boolean *value)
{
    boolean rv = *value;
    *value = TRUE;
    return rv;
}
```
- Example: `while (test&set(&lock)) ;`
 - It sets the `lock` variable to TRUE, at the same time, and checks whether its previous value was TRUE
 - if the previous value was TRUE, it does not go into critical section

Swap Instruction

- Definition:

```
void Swap (boolean *a, boolean *b)
{
    boolean temp = *a;
    *a = *b;
    *b = temp;
}
```

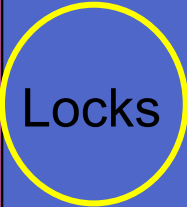
Solution using Swap

- Shared Boolean variable lock initialized to FALSE; Each process has a local Boolean variable key

- Solution:

```
do {  
    key = TRUE;  
    while ( key == TRUE)  
        Swap (&lock, &key );  
  
        // critical section  
  
    lock = FALSE;  
  
        // remainder section  
  
} while (TRUE);
```

Synchronization

Programs	Shared Programs
Higher-level API	 Locks Semaphores Monitors Send/Receive
Hardware	Load/Store Disable Ints Test&Set Comp&Swap

- We are going to implement various higher-level synchronization primitives using atomic operations
 - Need to provide primitives useful at user-level

Basic Structure for Synchronization

Thread 1



Sync. Entry

Critical
Section

Sync. Exit

Thread 2



Sync. Entry

Critical
Section

Sync. Exit

....

Thread n



Sync. Entry

Critical
Section

Sync. Exit

How to coordinate locks?

Thread 1

each thread checking out 5 locks?

Thread 2

Thread 3

Thread 4

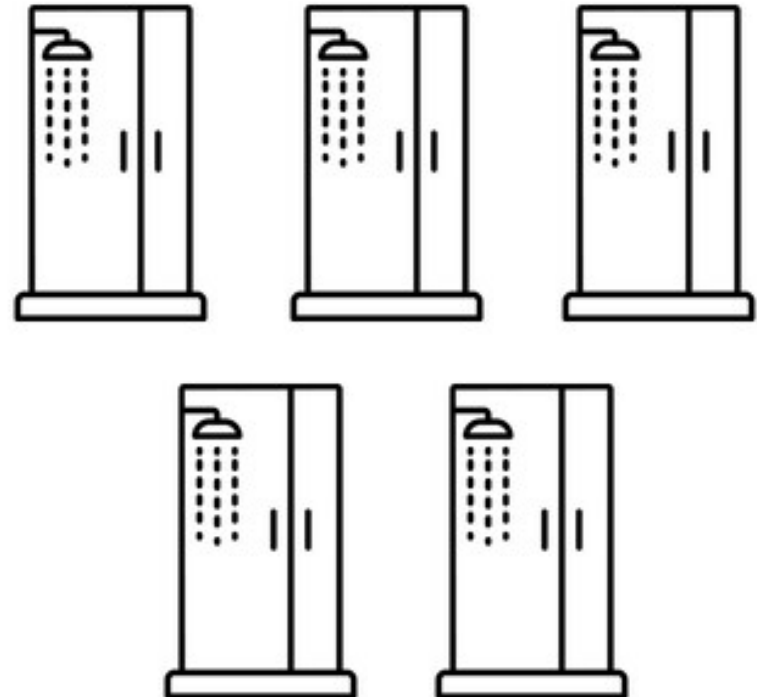
Thread 5

Thread 6

⋮

Thread 10

Shower Booths 1 .. 5



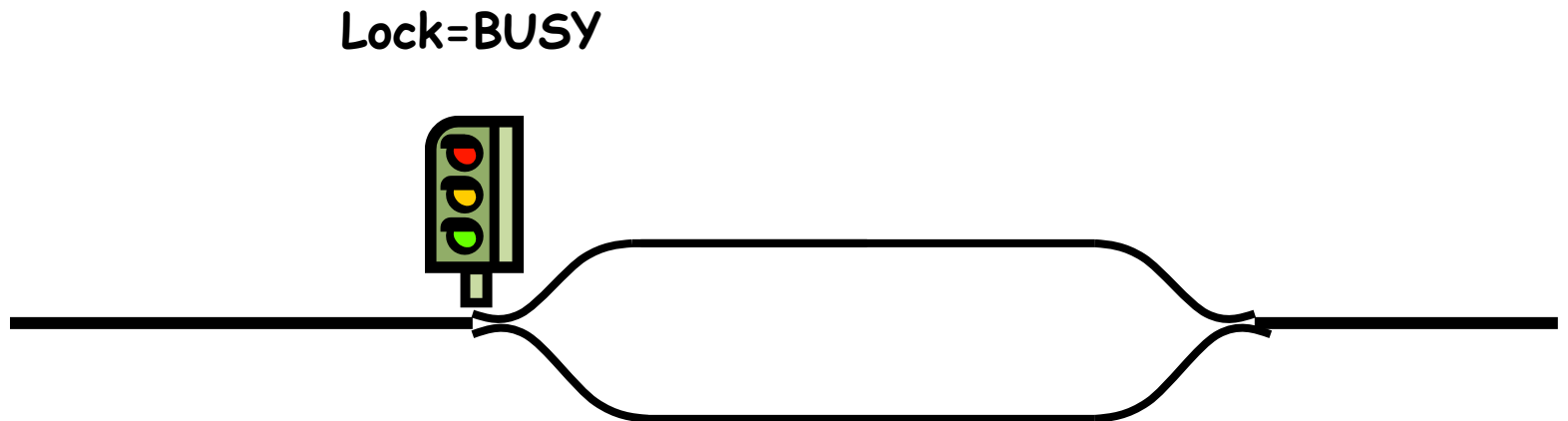
Synchronization

Programs	Shared Programs
Higher-level API	Locks Semaphores Monitors Send/Receive
Hardware	Load/Store Disable Ints Test&Set Comp&Swap

- We are going to implement various higher-level synchronization primitives using atomic operations
 - Need to provide primitives useful at user-level

Limitation of Lock?

- How about multiple threads entering critical sections?
 - Two trains (not only one) can get into rest areas



Semaphores



- Semaphores are a kind of generalized locks
 - First defined by Dijkstra in late 60s
 - Main synchronization primitive used in original UNIX
- Definition: a Semaphore has one variable and supports the following two operations:
 - *semaphore*: a non-negative integer value indicating how many resources (i.e., locks) are available
 - *wait()*: an atomic operation that waits for semaphore to become positive, then decreases it by 1
 - *signal()*: an atomic operation that increases the semaphore by 1, waking up a waiting thread, if any

Two Uses of Semaphores (1)

- Mutual Exclusion (initial value = 1)

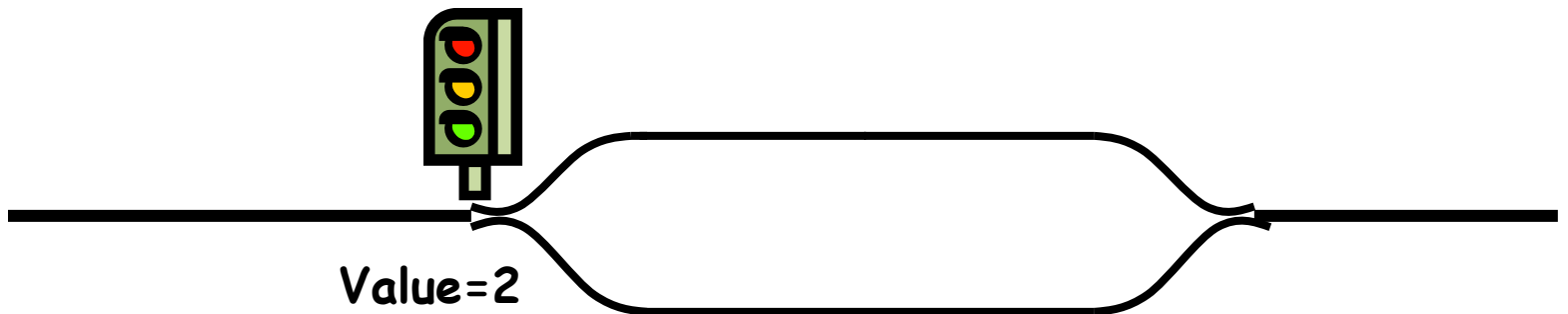
- Also called “Binary Semaphore”.
- Can be used for mutual exclusion:

```
semaphore.wait();  
// Critical section goes here  
semaphore.signal();
```

- **Counting** semaphore – integer value can range over an unrestricted domain

Semaphores

- Semaphore from railway analogy
 - Here is a semaphore initialized to 2 for resource control:



Semaphores

- Execution order dependency between threads
 - One train leaves only after another train comes



Two Uses of Semaphores (2)

- Scheduling Constraints (initial value = 0)
(Suppose we want thread 1 to do something only after thread 2 is done with something else)
 - Allow thread 1 to wait for a signal from thread 2, i.e., thread 2 **schedules** thread 1 when a given **constraint** is satisfied
 - Example: suppose you had to implement 'File Downloading & Processing', where Thread A must wait until Thread A has finished downloading the file:

Initial value of semaphore = 0

Called by
ThreadB

```
StartProcessing {  
    semaphore.wait();  
}
```

Called by
ThreadA

```
FinishDownlodng {  
    semaphore.signal();  
}
```



Semaphores

- Semaphore from railway analogy
 - Here is a semaphore initialized to 0 for scheduling constraint:
 - One train leaves only after another train comes



Value=0

Synchronization

Programs	Shared Programs
Higher-level API	Locks Semaphores Monitors Send/Receive
Hardware	Load/Store Disable Ints Test&Set Comp&Swap

- We are going to implement various higher-level synchronization primitives using atomic operations
 - Everything is pretty painful if only atomic primitives are load and store
 - Need to provide primitives useful at user-level

Recap: Synchronization

- Coordinates multiple threads to execute as we intended (according to its own synchronization semantics)

thread 1

```
synch.entry()  
...  
critical section;  
...  
synch.exit()
```

thread 2

```
synch.entry()  
...  
critical section;  
...  
synch.exit()
```

thread N

```
synch.entry()  
...  
critical section;  
...  
synch.exit()
```

- Synchronization semantics
 - mutual exclusion
 - what else?

Recap: Semaphore

- Counting semaphores (vs. binary semaphores)
 - how many units of a particular resource are available
- Execution order constraints (vs. mutual exclusion)
 - can enforce certain execution orders across threads
- Summary
 - semaphores are a useful tool in the prevention of race conditions; however, their use is by no means a guarantee that a program is free from these problems.

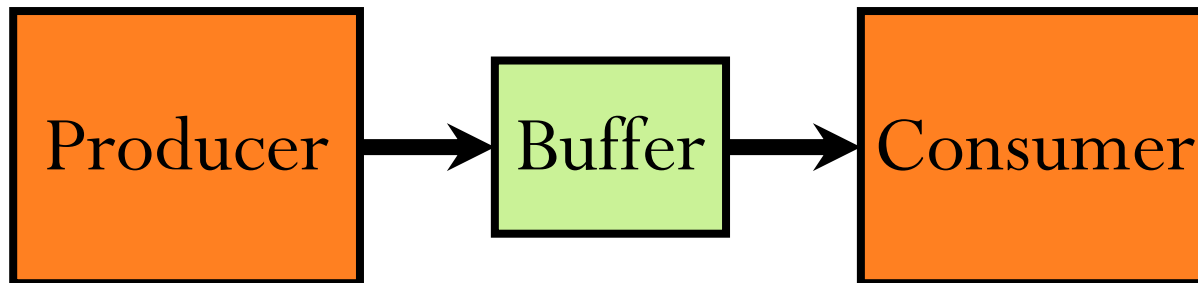
Classical Problems of Synchronization

- Producer-consumer Problem
- Readers and Writers Problem
- Dining-Philosophers Problem

Bounded-Buffer Problem

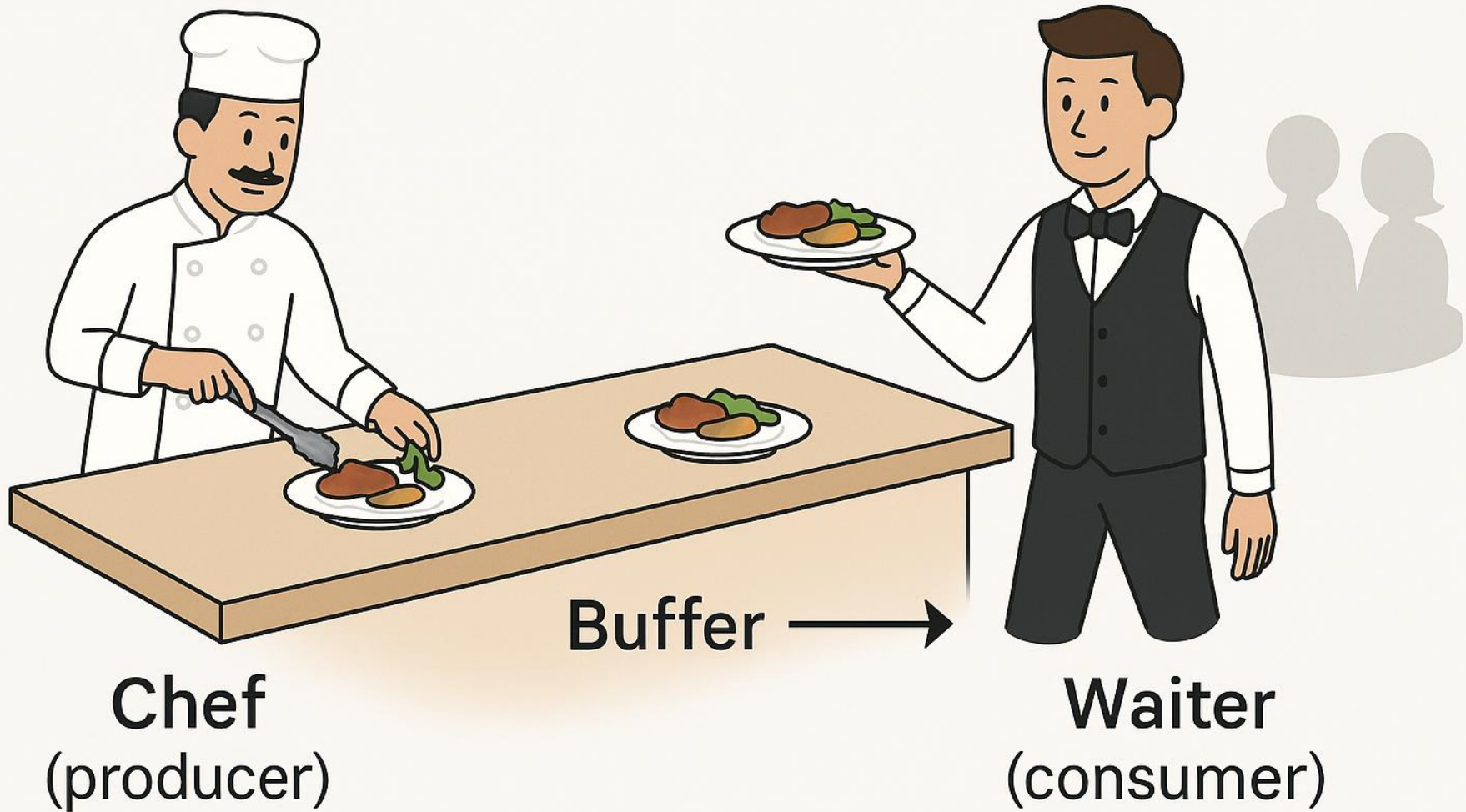
- Problem Definition

- Producer puts things into a shared buffer
- Consumer takes them out
- Need synchronization to coordinate producer/consumer



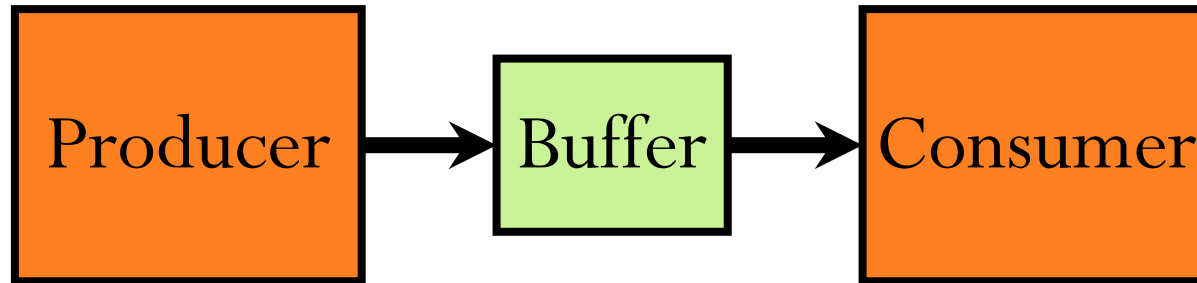
- Don't want producer and consumer to have to work in lockstep, so put a fixed-size buffer between them
 - Need to synchronize access to this buffer (mutual exclusion)
 - Producer needs to wait if buffer is full
 - Consumer needs to wait if buffer is empty

Real-Life Analogy



Solution to Bounded Buffer

- Producer & consumer threads



Producer

```
synch.entry()  
...  
// critical section  
Enqueue(item);  
...  
synch.exit()
```

Consumer

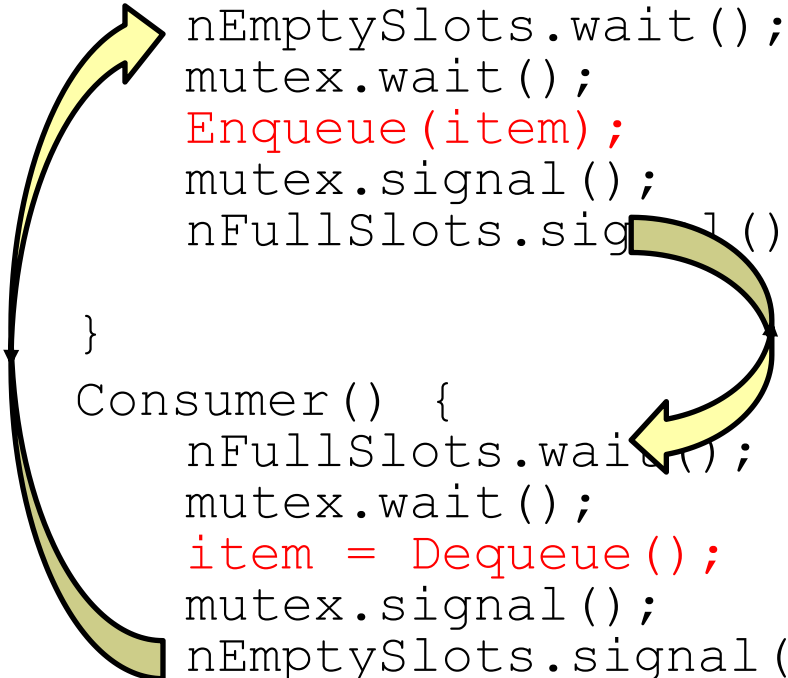
```
synch.entry()  
...  
//critical section  
item = Dequeue;  
...  
synch.exit()
```

Correctness constraints for solution

- Correctness Constraints:
 - Consumer must wait for producer to fill slots, if empty (execution order dependency)
 - Producer must wait for consumer to make room in buffer, if all full (execution order dependency)
 - Only one thread can manipulate buffer queue at a time (mutual exclusion)
- General rule of thumb:
Use a separate semaphore for each constraint
 - Semaphore nFullSlots; // consumer's constraint
 - Semaphore nEmptySlots; // producer's constraint
 - Semaphore mutex; // mutual exclusion

Full Solution to Bounded Buffer

```
Semaphore nFullSlots = 0; // Initially, no data
Semaphore nEmptySlots = N; // Initially, N empty slots
Semaphore mutex = 1;      // No one using machine
```



```
Producer(item) {
    nEmptySlots.wait(); // Wait until space
    mutex.wait();       // Wait until machine free
    Enqueue(item);
    mutex.signal();
    nFullSlots.signal(); // Tell consumers there is
                        // more data
}

Consumer() {
    nFullSlots.wait(); // Check if there's data
    mutex.wait();      // Wait until machine free
    item = Dequeue();
    mutex.signal();
    nEmptySlots.signal(); // tell producer need more
    return item;
}
```

Producer proceeds if any empty slot, and consumer does if any data in the buffer

Discussion about Solution

- Why asymmetry?

- Producer does: `nEmptySlots.wait()`, `nFullSlots.signal()`
- Consumer does: `nFullSlots.wait()`, `nEmptySlots.signal()`

Decrease # of
empty slots

Increase # of
occupied slots

Decrease # of
occupied slots

Increase # of
empty slots

Role	Waits on	Signals to
Producer	empty slots	full slots
Consumer	full slots	empty slots

Discussion about Solution

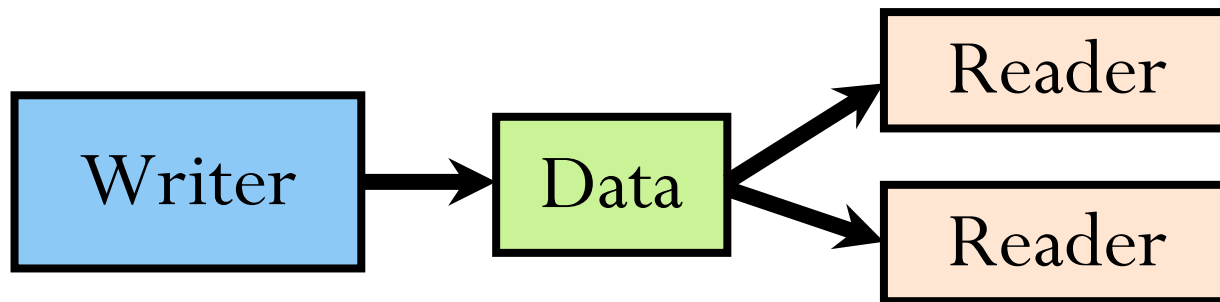
- Is order of wait's important?
- Is order of signal's important?
- What if we have 2 producers or 2 consumers?

```
Producer(item) {  
    mutex.wait();  
    nEmptySlots.wait();  
    Enqueue(item);  
    mutex.signal();  
    nFullSlots.signal();  
}
```

```
Consumer() {  
    nFullSlots.wait();  
    mutex.wait();  
    item = Dequeue();  
    mutex.signal();  
    nEmptySlots.signal();  
    return item;  
}
```


Readers-Writers Problem

- A data set is shared among a number of concurrent processes
 - Readers – only read the data set; they do **not** perform any updates
 - Writers – can both read and write
- Problem – allow multiple readers to read at the same time. Only one single writer can access the shared data at the same time



- Shared Data
 - Data set
 - Integer **nReaders** initialized to 0
 - Semaphore **wrt** initialized to 1 // for mutually exclusive access to Data
 - Semaphore **mutex** initialized to 1 // for mutual exclusion of *nReaders*

Readers-Writers Problem (Cont.)

- The structure of a writer process

```
do {  
    wait (wrt) ;  
  
    //  writing is performed  
  
    signal (wrt) ;  
  
} while (TRUE);
```

Readers-Writers Problem (Cont.)

- The structure of a reader process

do {

```
    wait (mutex) ;           // must synch with other readers, if any,  
    nReaders ++ ;           // to access the shared variable  
    if (nReaders == 1)       // if it is the first reader  
        wait (wrt) ;        // must synch with the writer
```

```
    signal (mutex)
```

```
    // reading is performed
```

```
    wait (mutex) ;  
    nReaders - - ;  
    if (nReaders == 0)  
        signal (wrt) ;
```

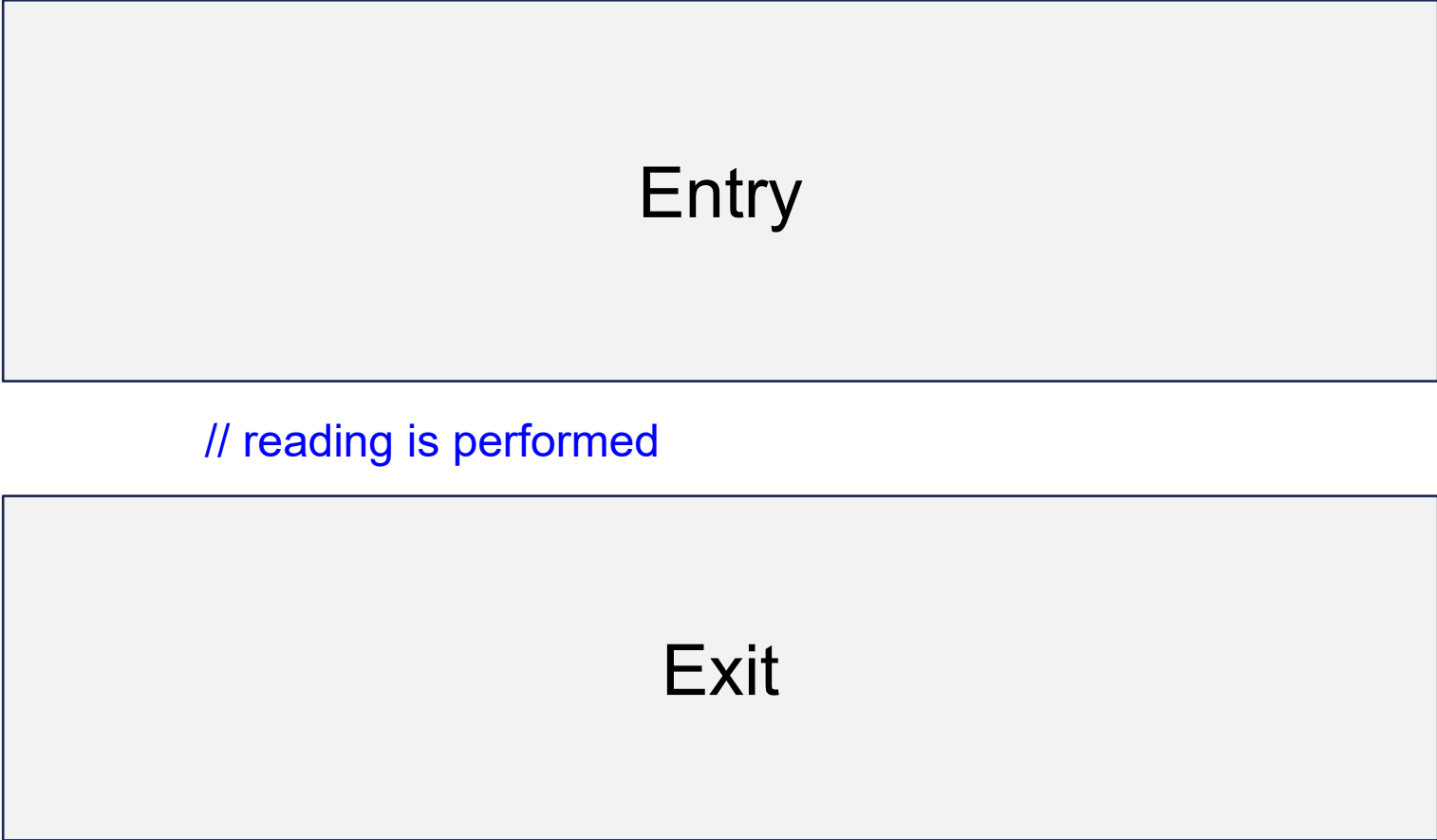
```
    signal (mutex) ;
```

```
} while (TRUE);
```

Readers-Writers Problem (Cont.)

- The structure of a reader process

do {



The diagram illustrates the structure of a reader process. It consists of two main rectangular blocks, one above the other, both with a light gray background and a dark blue border. The top block is labeled 'Entry' in the center. Below this block, the text '// reading is performed' is centered. The bottom block is labeled 'Exit' in the center. The entire structure is enclosed within a blue code block, with 'do {' at the top left and '} while (TRUE);' at the bottom left.

Entry

// reading is performed

Exit

} while (TRUE);

Readers-Writers Problem (Cont.)

- The structure of a reader process

do {

wait (wrt) ;

// reading is performed

signal (wrt) ;

} while (TRUE);

Readers-Writers Problem (Cont.)

- The structure of a reader process

do {

 wait (mutex) ; // must synch with other readers, if any,

 nReaders ++ ; // to access the shared variable

 if (nReaders == 1) // if it is the first reader

 wait (wrt) ; // must synch with the writer

 signal (mutex)

 // reading is performed

 wait (mutex) ;

 nReaders - - ;

 if (nReaders == 0)

 signal (wrt) ;

 signal (mutex) ;

} while (TRUE);